



DATA STRUCTURES

CONTENTS



24. Insertion and Deletion – AVL Trees

T.V. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept AVL, a balanced binary search tree. In this module we will discuss the insertion and deletion operations of AVL trees and the rebalancing carried out for re-establishing the balance while maintaining the binary search property.

Learning Objectives

The learning objectives of the module are as follows:

- To understand the Insertion operations of AVL Trees
- To discuss Deletion from AVL Trees
- To Outline the Pros and Cons of AVL Trees

24.1 Insertion into AVL Trees

The first step of insertion of a node into the AVL tree is a normal insertion using a BST insertion algorithm. However in addition we have to rebalance the tree if an imbalance occurs. As we have discussed in the last module, an imbalance occurs if a node's balance factor changes from -1 to -2 or from +1 to +2. Rebalancing is done at the deepest or lowest unbalanced ancestor of the inserted node.

There are three cases that can happen during insertion:

1. Insertion that does not cause an imbalance.
2. Same side (left-left or right-right) insertion that causes an imbalance. This type of insertion requires a single rotation to rebalance.
3. Opposite side (left-right or right-left) insertion that causes an imbalance. This type of insertion requires a double rotation to rebalance.

24.2 Insertion Algorithm

The first step of the insertion algorithm for AVL trees is the same as insertion into a binary search tree. Here we first find a place for the value to be inserted, and then insert it. Now comes the next step of insertion into the AVL tree which is searching back from the inserted node looking for imbalance. If there is an imbalance when a new element is added as the outside grandchild (that is the left grandchild of a left child (left-left) or the right grandchild of a right child (right-right)) (Figure 24.1 (a)) we perform single rotation and exit. When a new item is added as an inside grandchild (Figure 24.1 (b)), the imbalance is fixed with a double right or left rotation. As already discussed in the previous module an inside grandchild is when we have left grandchild of a right child (left-right) or the right grandchild of a left child (right-left)).

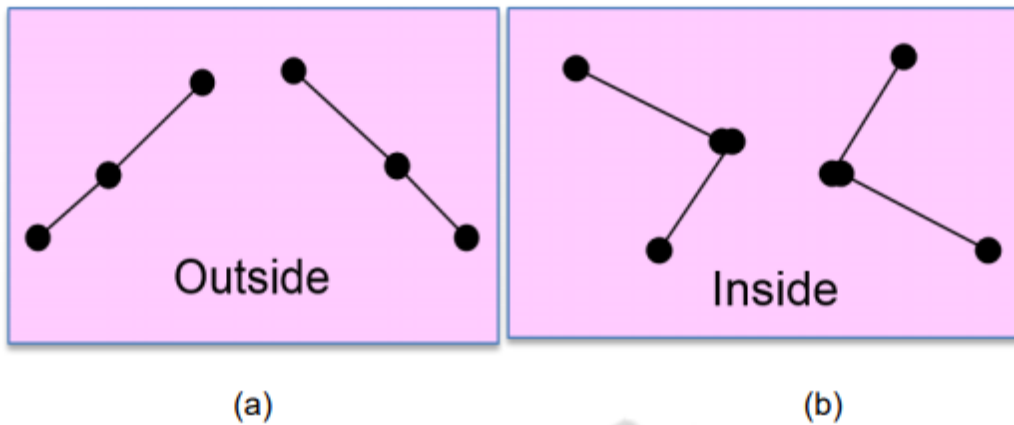


Figure 24.1 Outside and Inside Grandchildren

As we have already discussed in the previous module the insert operation may cause balance factor to become $+2$ or -2 for some node. Now only nodes on the path from the insertion point to the root node have possibly changed in height. Therefore after insertion, we go back up to the root node by node, updating heights as we go. If a new balance factor (the difference $h_{\text{left}} - h_{\text{right}}$) is $+2$ or -2 , we need to adjust the balance of the tree by *rotation* around the node. Now let us consider a valid AVL sub-tree (Figure 24.2). Before the insertion into the subtree M the tree was a valid AVL tree, but after the insertion the AVL property is violated at node j where the balance factor has become $+2$.

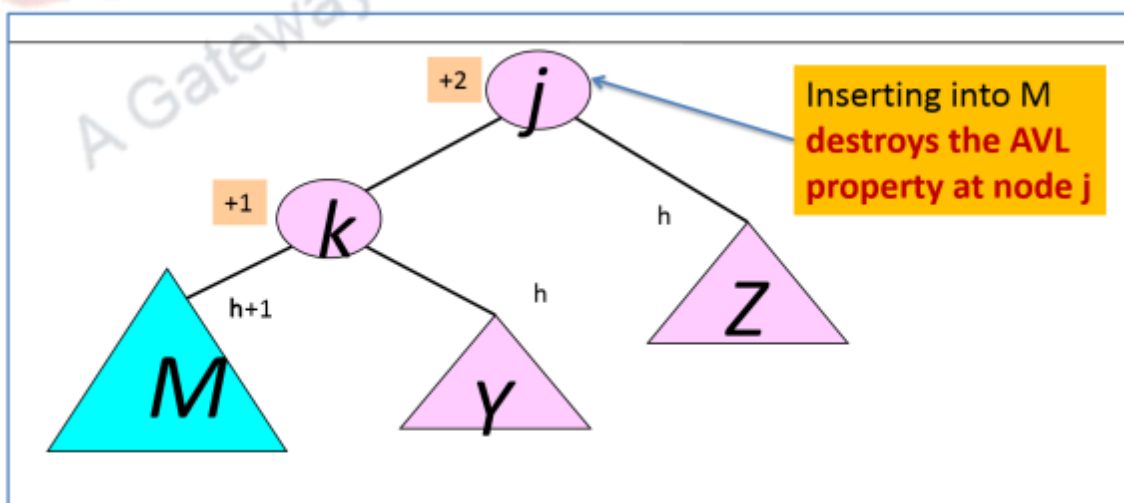


Figure 24.2 Insertion into an AVL tree

24.3 Rebalancing

Rebalancing is done at the deepest unbalanced ancestor of the inserted node. Now let the node that needs rebalancing be denoted as X. The rebalancing is performed through four separate types of rotations as given below:

Outside Cases (require single rotation) :

1. Insertion into left subtree of left child of X – **(Single Right Rotation)**
2. Insertion into right subtree of right child of X – **(Single Left Rotation)**

Inside Cases (require double rotation) :

3. Insertion into right subtree of left child of X – **(Left-Right Rotation)**
4. Insertion into left subtree of right child of X – **(Right-Left Rotation)**

24.3.1 Insertion into left sub-tree of left child of X

Let us consider the **first case of insertion into left sub-tree of left child of X** (Figure 24.3). Here we have inserted 7 as left sub-tree of left child (8) of X (9). It is at X that the balancing is violated with balance factor of X becoming +2. Now this node X is the pivot. This imbalance occurred when the new element (7) was added to the left sub-tree of the outside left grandchild that is we have the case of **single right rotation**.

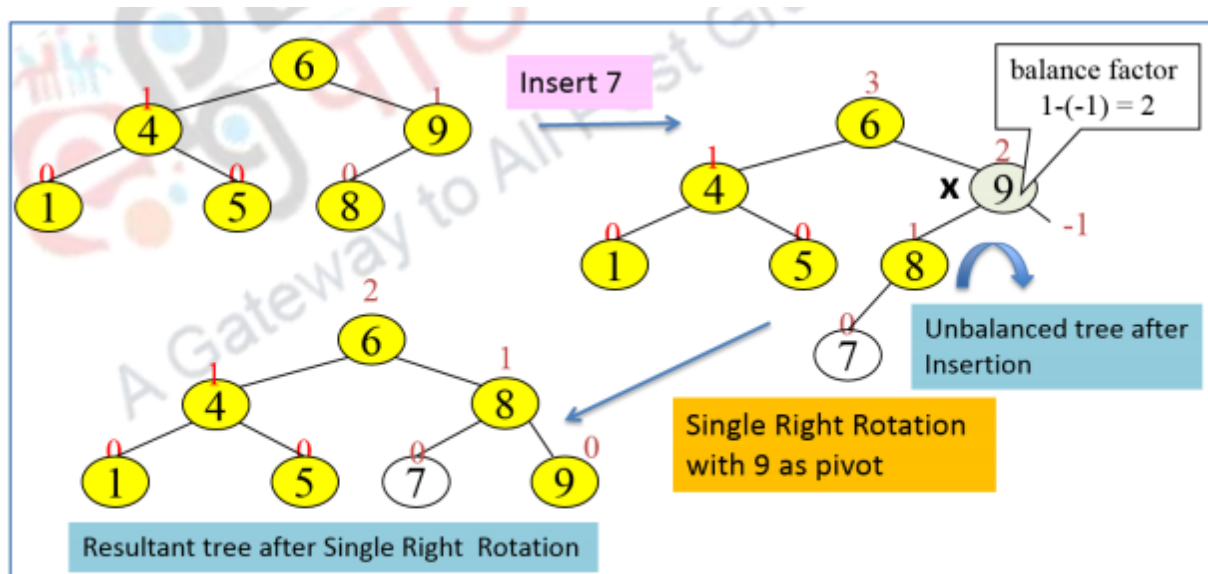


Figure 24.3 Insertion into left sub-tree of left child of X

Now we carry out single right rotation of 8 about 9. In this case 8 becomes the right child of the parent (6) of 9 while 9 becomes the right child of 8. The BST property of the AVL tree is now maintained.

24.3.2 Insertion into right sub-tree of right child of X

Let us consider the **second case of insertion into right sub-tree of right child of X** (Figure 24.4). Here we have inserted 45 as right sub-tree of right child (40) of X (35). It is at X that the balancing is violated with balance factor of X becoming -2. Now the node X is the pivot. This imbalance occurred when the new element (45) was added to the right sub-tree of the outside right grandchild that is we have the case of **single left rotation**. Now we carry out single left rotation of 40 about 35. In this case 40 becomes the right child of the parent (30) of 35 while 35 becomes the left child of 40. The BST property of the AVL tree is now maintained.

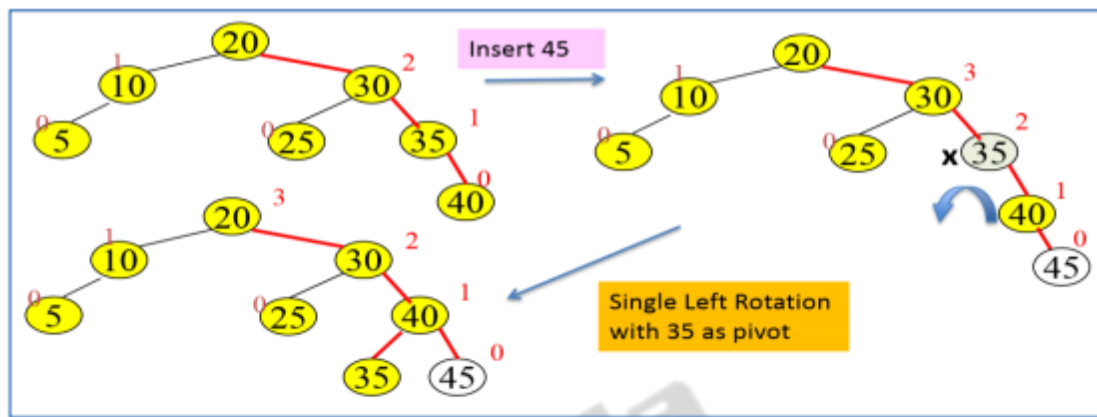


Figure 24.4 Insertion into right sub-tree of right child of X

24.3.3 Insertion into left sub-tree of right child of X

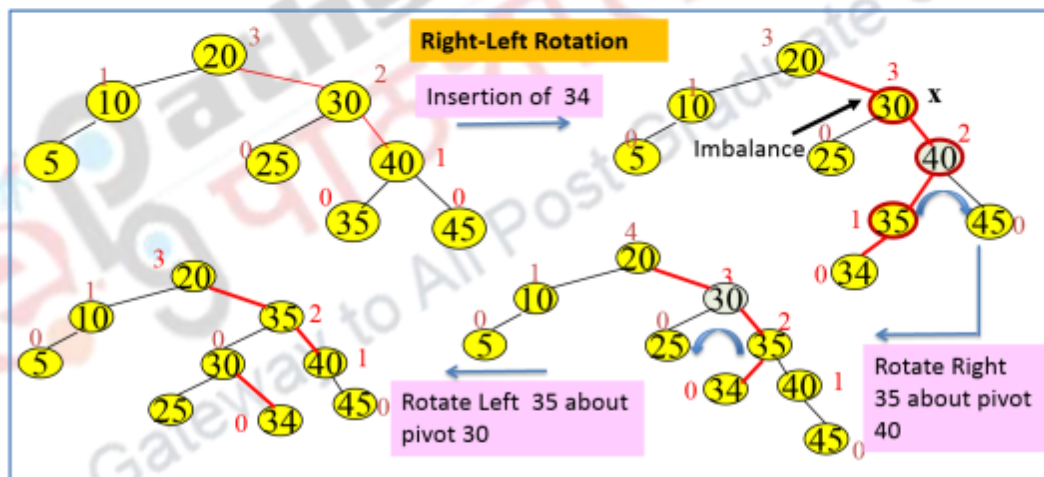


Figure 24.5 Insertion into left sub-tree of right child of X

Let us consider the **third case of insertion into left sub-tree of right child of X** (Figure 24.5). Here we have inserted 34 as left sub-tree of right child (40) of X (30). Balance is violated at X balance factor becoming -2. **Now we carry out two rotations that is a right rotation followed by a left rotation:**

- The first is a **single right rotate** of 35 about the first pivot (40). Here 35 becomes the new right child of X while 40 becomes the right child of 35.
- Next we carry out a **single left rotate** of 35 about second pivot node X (30). In this case 35 becomes the left child of the parent (20) of 30 while 30 becomes the new left child of 35. The BST property of the AVL tree is now maintained.

24.3.4 Insertion into right sub-tree of left child of X

Let us consider the **fourth case of insertion into right sub-tree of left child of X** (Figure 24.6). Here we have inserted 7 as right sub-tree of left child (5) of X (10). Balance is violated at X balance factor becoming +2. When a new item (7) is added to the sub-tree of the inside grandchild, the imbalance is fixed with a double rotation. Now we carry out two rotations that is a **left rotation followed by a right rotation**:

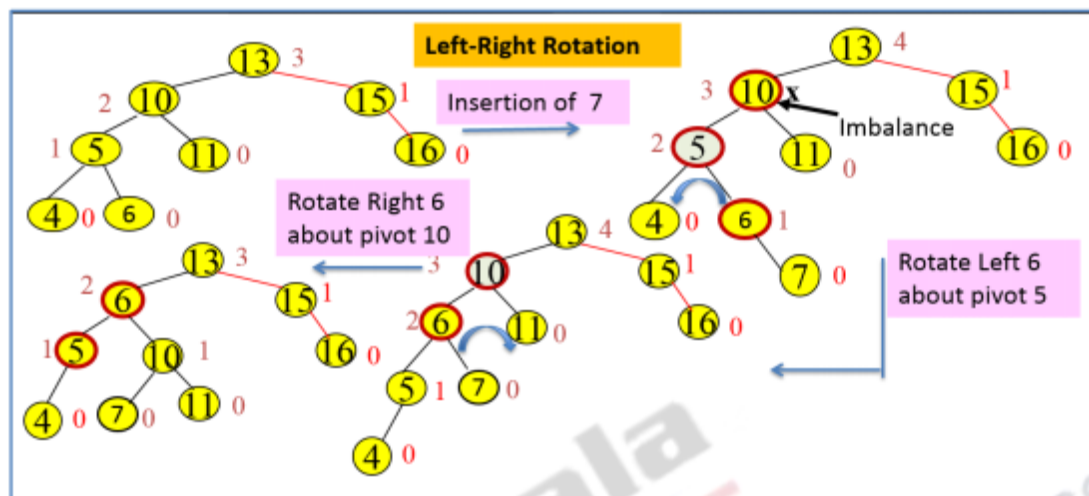


Figure 24.6 Insertion into right sub-tree of left child of X

AvlTree **Insert**(ElementType X, AvlTree T){ if(T == NULL){

/* Create and return a one-node tree */

 T = malloc(sizeof(struct AvlNode));

 if(T == NULL)

 FatalError("Out of space!!!");

 else {

 T->Element = X; T->Height = 0;

 T->Left = T->Right = NULL;

 }

 }

 else if(X < T->Element){

 T->Left = Insert(X, T->Left); **/* Insertion at the left */** if(Height(T->Left) -
Height(T->Right) == 2) if(X < T->Left->Element)

 T = SingleRotateWithLeft(T); **/* LL */**

 else

 T = DoubleRotateWithLeft(T); **/* LR */** } else if(X > T->Element){

 T->Right = Insert(X, T->Right); **/* Insertion at the right */** if(Height(T->Right) -
Height(T->Left) == 2) if(X > T->Right->Element)

 T = SingleRotateWithRight(T); **/* RR */**

 else


```

T = DoubleRotateWithRight( T ); /* RL */

} /* Else X is in the tree already; we'll do nothing */ T->Height=Max(Height(T-
>Left),Height(T->Right))+1;

return T;

}

```

Figure 24.7 The Insertion Algorithm

- The first is a **single left rotate** of 6 about the first pivot (5). Here 6 becomes the new left child of the parent (10) of 5 while 5 becomes the left child of 6.
- Next we carry out a **single right rotate** of 6 about the second pivot node X (10). In this case 6 becomes the left child of parent (13) of X (10) while 10 becomes the new right child of 6. The BST property of the AVL tree is now maintained

The detailed algorithm for insertion is given in Figure 24.7.

24.4 Deletion

The first step in deletion is deleting the node to be deleted X as in an ordinary binary search tree. Note that whatever may be the cases of deletion from the binary search tree, the last node deleted is a leaf. The deletion may result in an imbalance. Now we follow the path from the new leaf towards the root. For each node X encountered, we need to check if heights of left(X) and right(X) differ by at most 1. If yes, proceed to the parent(X) which now becomes the new X. If not, that is the heights differ by more than 1 then we need to perform an appropriate rotation at X. There are 4 cases as in the case of insertion. In the case of deletion, after we perform a rotation at X, we may have to perform a rotation at some ancestor of X. Thus, we must continue to trace the path until we reach the root. Now we will discuss the case of deletion where no balancing is needed and the cases where rebalancing of the tree is needed when an imbalance occurs after deletion.

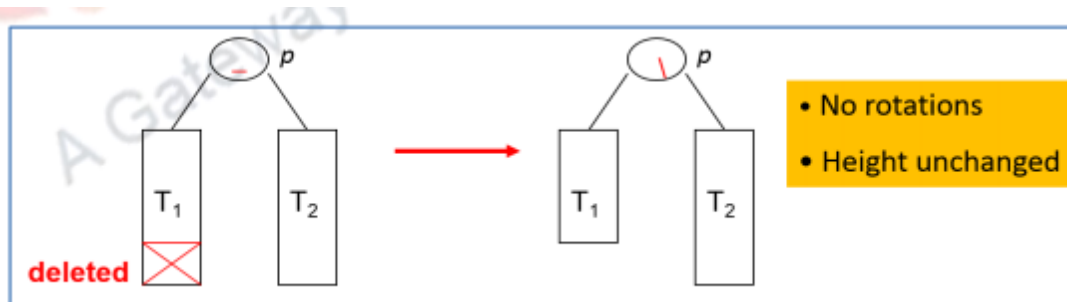


Figure 24.8 Deletion with no Imbalance with node P having balance factor=0

We can consider three cases for deletion:

1. Deletion that does not cause an imbalance.
2. Deletion that requires a single rotation to rebalance.
3. Deletion that requires two or more rotations to rebalance.

24.4.1 Deletion with no Imbalance

Let us consider the simplest case of deletion that is deletion which causes no imbalance. We first consider the current node p that has sub-trees T_1 and T_2 with equal heights and so the **balance factor of p is zero** (Figure 24.8). When deletion occurs say in the left sub-tree T_1 , its height is decreased but the height of p remains unchanged. The balance factor of p becomes (-1) . This is allowed so there is no imbalance. This is illustrated using the example given in Figure 24.9. The deletion of 14 causes the balance factor of 15 to change from 0 to -1 but its height remains the same as before deletion and hence no rotation is needed.

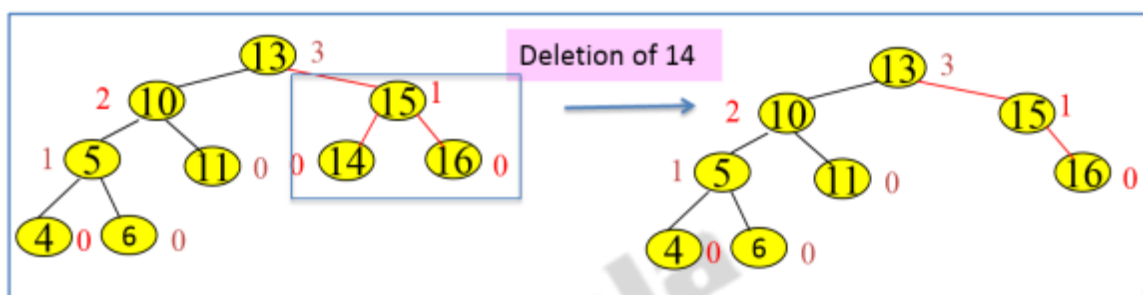


Figure 24.9 Deletion with no Imbalance for the case in Figure 24.8

We next consider the current node p whose balance factor is not 0. Let us see the case where balance factor of p is +1 (Figure 24.10) and the taller subtree (here T_1) was shortened. Now the balance factor of p becomes 0, the height of the tree is reduced but there is no imbalance and hence there is no need for rotations.

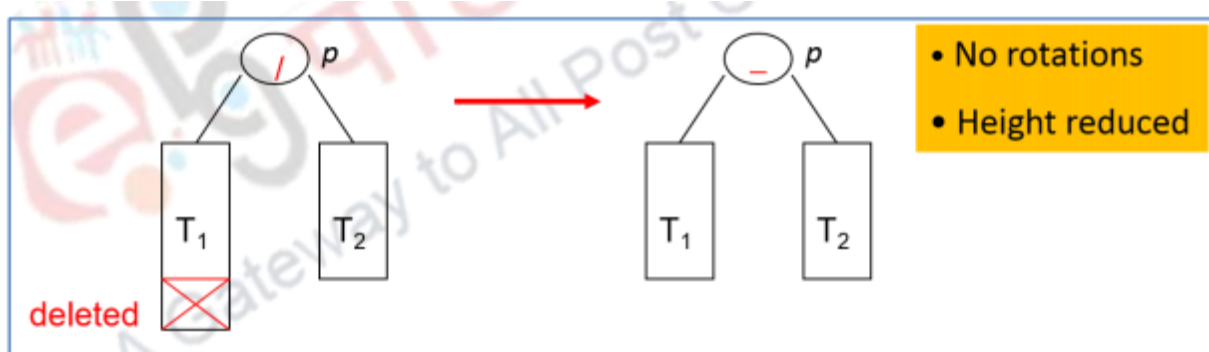


Figure 24.10 Deletion with no Imbalance with node p having balance factor $=+1$

24.4.2 Deletion with Single Rotation

Now let us consider the case shown in Figure 24.11. **The balance factor of q is 0.** Before deletion the left sub-tree of p had height h and the right sub-tree of p had height $=h+1$. Now we delete a node from left sub-tree such that its height become $h-1$. Now the balance factor at p changes from -1 to -2 and the AVL condition is violated. Then we need to carry out rebalancing. In this case we need to carry out a single right rotation of q about p where p becomes the left child of q and q 's left child becomes p 's right child.

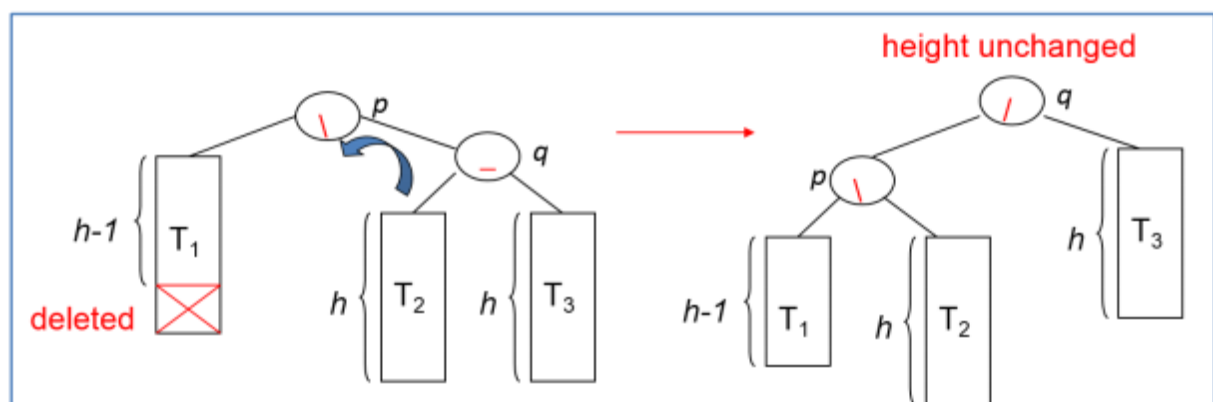


Figure 24.11 Deletion with Single Rotation where the balance factor at q is 0

Now let us consider the case shown in Figure 24.12. **The balance factor of q is equal to that of p.** Before deletion left sub-tree of p from T1, the balance factor of p becomes $(h-1) - (h+1) = -2$ and hence there is an imbalance. Now we do a single right rotation of q about P where P becomes left sub-tree of q and T2 becomes right sub-tree of P. The overall height of the tree is reduced.

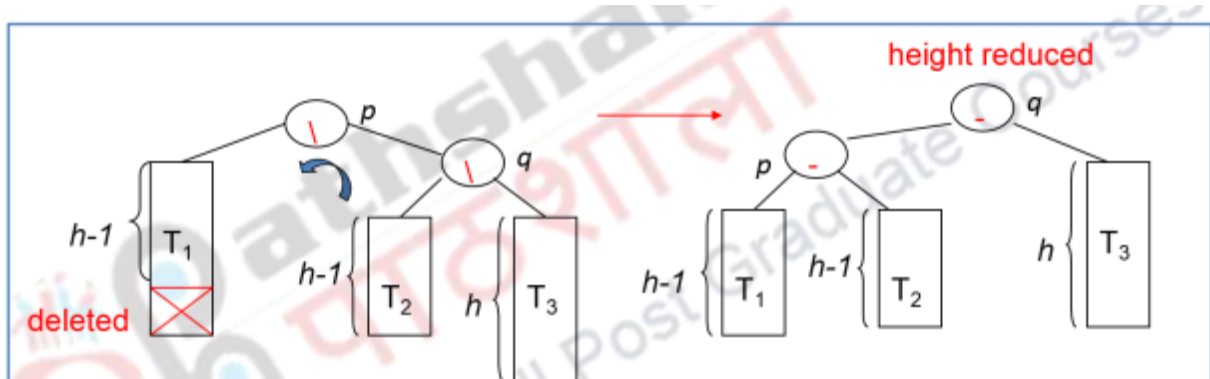


Figure 24.12 Deletion with Single Rotation- balance factor of p and q are equal

24.4.2.1 Deletion with Single Right Rotation

Now let us consider the example of deletion which causes an imbalance and leads to single right rotation (Figure 24.13). Here we assume that 40 is deleted (From T3 of Figure 24.12). This causes an imbalance at node 35 (q). Now to rebalance we carry out single right rotation of 32 (T2) with 35 (q) as the pivot. Now 32 becomes the right child of the parent (30) of 35 while 35 becomes the right child of 32.

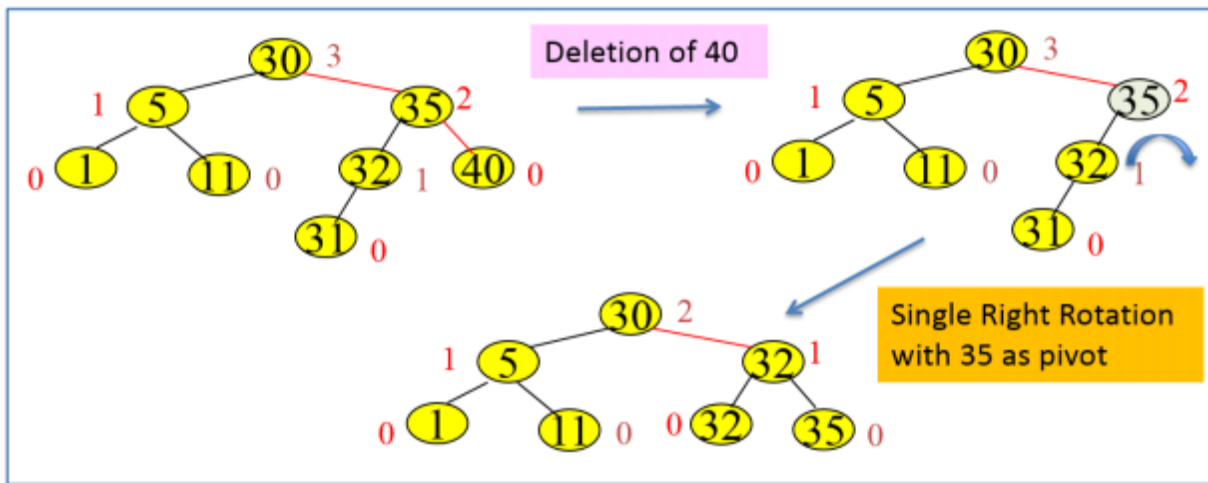


Figure 24.13 Deletion with Single Right Rotation

24.4.2.2 Deletion with Single Left Rotation

Now let us consider the example of deletion which causes an imbalance and leads to single left rotation (Figure 24.14). Here we assume that 32 is deleted. This causes an imbalance at the root node 44. Now to rebalance we carry out single left rotation of 62 with 44 as the pivot. Now 62 becomes the new root node while 44 becomes the right child of 62. The right sub-tree of 62 rooted at 50 now becomes the right sub-tree of 44.

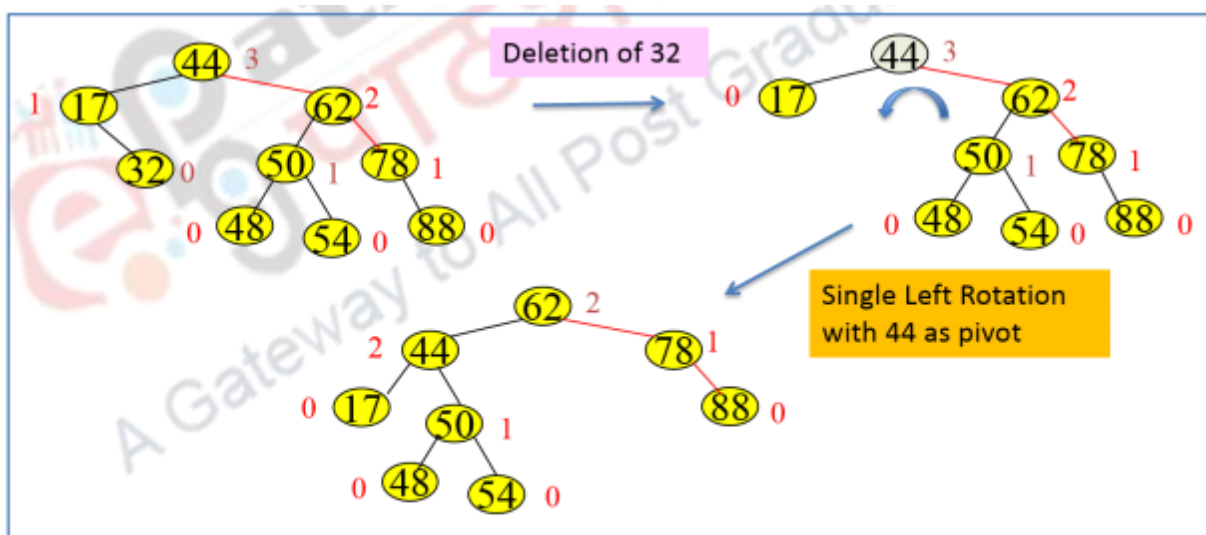


Figure 24.14 Deletion with Single Left Rotation

24.4.3 Deletion with Double Rotation

In case the balance factors of p and q are opposite then we need to apply a double rotation (Figure 24.15). The balance factor at q will be 0 or 1. The balance factor of node p will be 0 or -1. Now let us assume p 's left child has height h before deletion. Let us assume that the right child q of p has left sub-tree rooted at r to be of height $h-1+1=h$ or $h-2+1=h-1$. The left sub-tree of q has height $h-1$. Now let us assume that a node is deleted from the left sub-tree of p making its height $h-1$. We first right rotate r about q and then left rotate r about p . Then we set the balance factors of the new root r to be 0.

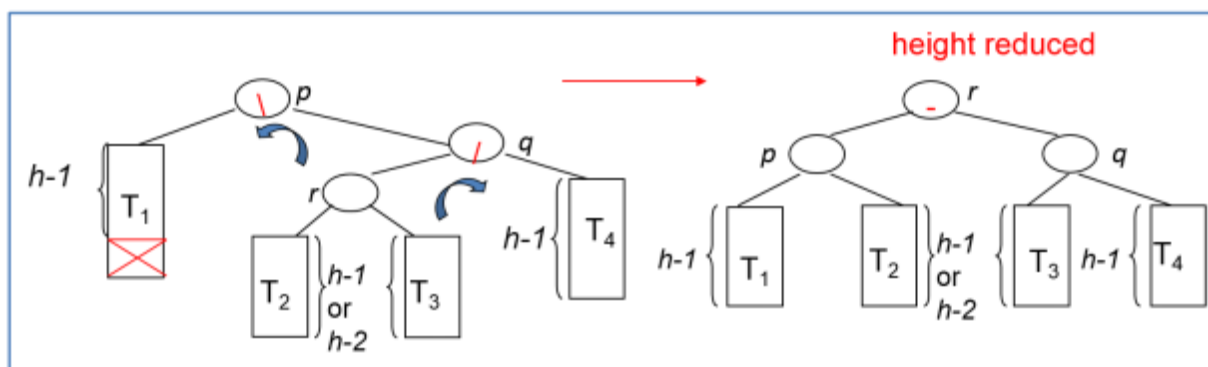


Figure 24.15 Deletion with two rotations

For the case of deletion with two or more rotations let us consider the example given in Figure 24.16. The deletion of 40 first causes an imbalance to occur at 35. Now we carry out a right rotate of 32 with 35 as pivot. This causes 32 to become right child of parent (30) of 35 and 35 itself becomes right child of 32. However this rotation causes an imbalance at the root node 30. This requires us to carry out another right rotation of 20 about the pivot 30. This causes 20 to become the new root and the right sub-tree of 20 becomes the right sub-tree of 30. Now the tree is balanced. Please note that in the case of deletion we need to check for imbalance and carry on rebalancing until the tree is balanced. This may cause more than two rotations in some situations.

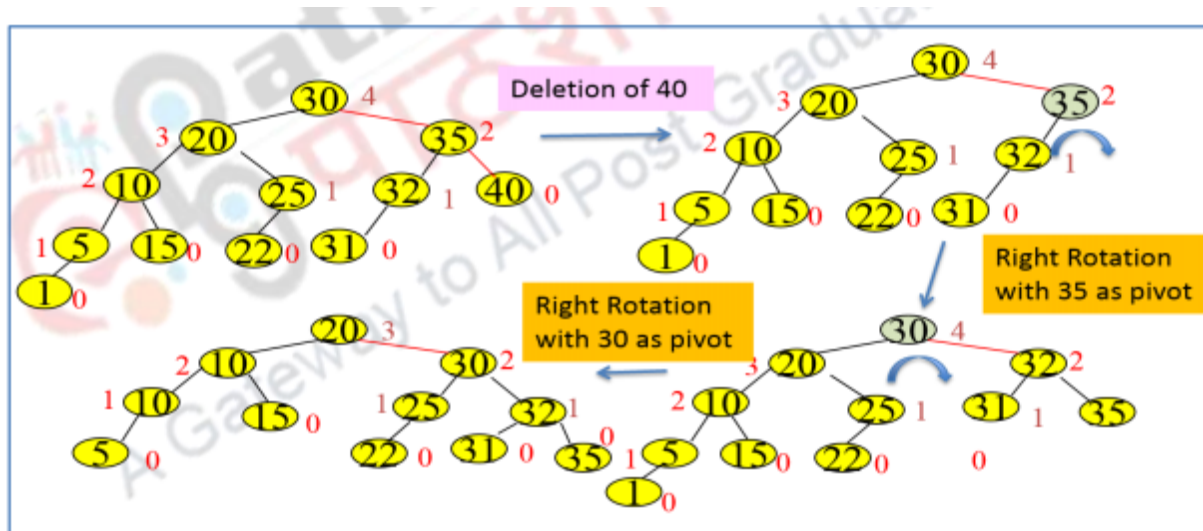


Figure 24.16 Example Deletion with two rotations

```
AvlTree Delete( ElementType X, AvlTree T ){ if( T == NULL ) Error("Item not Found);
```

```
    else if( X < T->Element ){
        T->Left = Delete( X, T->Left );
```

```
        if( Height( T->Left ) - Height( T->Right ) == -2 )/*Imbalance due to insertion*/
        if( Height(T->Right->Right) > Height(T->Right->Left) )
```

```
            T = SingleRotateWithRight( T );/* RR */
        else
            T = DoubleRotateWithRight( T );    /* RL */
    }
    else if( X > T->Element ){
        T->Right = Delete( X, T->Right );
```

```
        if( Height( T->Right ) - Height( T->Left ) == -2 )
            if( Height(T->Left->Left) > Height(T->Left->Right) )
                T = SingleRotateWithLeft( T );/* LL */
        else
            T = DoubleRotateWithLeft( T );    /* LR */
        else if( T->Left && T->Right ){ /* Found with two children */
            /* Replace with smallest in right subtree */
            TmpCell = FindMin( T->Right );
            T->Element = TmpCell->Element;
```

```
            T->Right = Delete( T->Element, T->Right ); if( Height( T->Right ) - Height( T->Left ) == -2 )
```

```
                if( Height(T->Left->Left) > Height(T->Left->Right) )/*LL*/
```

```
                    T = SingleRotateWithLeft( T );
```

```
        else
```

```
            T = DoubleRotateWithLeft( T );    /*LR*/
        }
```

```
        else {/* Found with one or zero child */
```

```
TmpCell = T;
```

```
T = T->Left ? T->Left : T->Right; /* Also handles 0 child */ free( TmpCell );
```

```
}
```

```
if( T!= NULL )
```

```
T->Height=Max(Height(T->Left), Height(T->Right))+1; return T;
```

```
}
```

Figure 24.17 The Deletion Algorithm

The details of the deletion algorithm is given in Figure 24.17.

24.5 Pros and Cons of AVL Trees

24.5.1 Arguments for AVL trees

The main advantage of the AVL tree is that it has been so designed that the search time for a node is of the $O(\log N)$ for a tree with N nodes since AVL trees are **always balanced**. Insertions and deletions are also of the $O(\log n)$. The height balancing needs for rebalancing during insertion and deletion increases the time by only a constant factor.

24.5.2 Arguments against using AVL trees

In general AVL trees are difficult to program and debug and moreover additional space is required to store the balance factor. Even though asymptotically the speed of the operations is faster, rebalancing does cost time. Most large searches are done in database systems on disk and use other structures such as B-trees and not AVL trees. It may sometimes be alright to have $O(N)$ for a single operation in case the total run time for many consecutive operations is fast which is the basis of Splay trees. Both B-trees and Splay trees will be discussed in subsequent modules.

Summary

- Explained the Insertion operations of AVL Trees with illustrative examples

- Discussed Deletion from AVL Trees with illustrative examples
- Outlined the Pros and Cons of AVL Trees

you can view video on Insertion and Deletion – AVL Trees



Web Links
https://courses.cs.washington.edu/courses/cse373/04wi/.../lecture08.ppt
https://www.cpp.edu/~ftang/.../trees/AVL%20tree%20animation.ppt
www.cs.fsu.edu/~asriniva/courses/DS14/lectures/Lec17-19.ppt
www.uwgb.edu/breznayp/cs351/slides/avl.ppt
www.cs.nyu.edu/courses/fall06/V22.0102-001/.../AVLTree-102-fa06.ppt
homes.ieu.edu.tr/~skondakci/.../CE221_week_11_Chapter4_TreesPart2
Supporting & Reference Materials
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014